Google Summer of Code 2026

Project Proposal: Canonical Master Release Track based on the complete Chroma-Fingerprint of the track

Applicant:
Ayush Sah

Organization:
Mixxx DJ Software

Possible Mentor:
Evelynne, Daniel

Contents

Introduction

I am Ayush Sah, a pre-final year undergraduate studying I.T. from IIIT Gwalior, India. I go by xARSENICx on github. As an aspiring Systems Engineer, I am driven by a deep fascination with how complex software interacts with hardware, a passion that manifests in my hobbies of low-level programming and audio engineering with applied Machine Learning.

My deep-rooted connection to music began at the age of 8, and over the years, I have formalised this passion with a Grade 7 ABRSM certification in Piano and a Grade 5 ABRSM certification in Music Theory from the Royal School of Music, London. I am an avid listener of all genres, from classical (Tchaikovsky, you beauty!) to experimental electronic (an Aphex Twin junkie), and my aspiration for music is boundless. Looking ahead, I aspire to merge my technical expertise with my creative vision to become a hobbyist music producer, crafting sounds that resonate as deeply as the pieces I first learned on the piano.

My technical journey is rooted in high-performance computing and systems design. I have a strong background in Competitive Programming (writing algorithmic code in C++ for the last 3.5 years), having achieved the Candidate Master title on Codeforces and 5-Star status on CodeChef (2000+). This algorithmic rigor translates directly into my development work, allowing me to breakdown complex tasks into small milestones and executing working plans to tackle them all.

I am a developer who prefers to let his work speak for itself and leave a tangible impact. Having already integrated myself into the Mixxx workflow, I am committed to the project for the long term and aspire to eventually contribute as a core developer, ensuring the software evolves alongside the needs of the global DJ community. I see GSoC 2026 as the ideal catalyst for this journey and am eager to dedicate my summer to Mixxx.

My Contributions

To become familiar with the Mixxx codebase and development workflow, I have actively engaged with the community by addressing various issues and implementing new features. Below is a summary of my contributions to Mixxx on GitHub:

Sl.	PR	Title	Summary
1	#15861 •	Add global analysis progress percentage & fix UI jitter	My first PR in Mixxx, I got familiar with pre-commit and took reviewers' feedback to improve upon the UX.
2	#15862 •	Implement configurable 'Recent Days' filter for Analyze view	Learned to work with Qt specific `.ui` files.
3	#15876 •	Tango: Implement 'Track has no STEMs' logic and fix alignment	My first work related to STEMs and skins, I learned to work with XML and git rebasing.
4	#15877 •	Deere: Implement 'Track has no STEMs' logic and fix alignment	Did the same changes as Tango, but for Deere.
5	#15885 •	Fixes Stem Control Alignment for long stem names	Another UX fix to improve upon the overall feel of the app.
6	#15887 •	Implement Stem VU Meter Control Objects	Created new backend COs connecting audio engine to skin meters. First time working on COs.
7	#841 •	docs: Add Stem VU Meter controls to appendix	Documented the new Stem VU meter control objects in the Mixxx user manual appendix. Resolved a non-trivial multi-branch merge conflict during the process.
8	#15898 •	feat(library): Add customizable date format option for Library and History	Worked with Qt Locale APIs to implement user-configurable date formatting.
9	#15919 •	Fix crash in AudioUnit backend due to off-by-one error in parameter syncing	First bug fix- patched an off-by-one bounds error in parameter sync. This one line fix took a very deep exploration of the codebase.
10	#15927 •	feat(preferences): Add unsigned int support to ConfigObject	Added some C++ support and wrote my first unit tests.

11	#15934 ●	feat: Add option to restore last used library view	An ongoing work along mxmilkiib.
12	#15970 ●	fix: prevent repeated load operation on QmlPlayerProxy	My first work on QML- I wish to support this project and see it become tangible as per the milestone.
13	#15974 ●	feat(qml): Implement scratch functionality for Spinny control	Implemented interactive scratching in the new QML engine and learnt about Mixxx's rendergraph rendering.
14	#16058 ●	build: Fix invalid `elseif` syntax in Apple platform checks	Resolved a CMake syntactical error causing errors during builds on MacOS.
15	#16104 ●	Fix AudioUnit thread exhaustion crash on startup (macOS)	I am actively developing macOS expertise with the goal of contributing macOS support, helping fill the current gap of macOS developers in the community.
16	#16106 ●	[2.5] Fix AudioUnit startup crash by loading out-of-process (macOS)	Another bugfix for MacOS, directly collaborating with issue raiser.
17	#16113 ●	Fix crash on exit due to invalid QLabel buddy references	Fixed problematic UI files directly, resolving exit crashes.
18	#16118 ●	build: Add pre-commit hook to detect invalid QLabel buddy properties	Wrote a static analysis Python script for UI files to add pre-commit hooks.
19	#16119 ●	Fix `--settings-path` accessibility handling across all platforms	Navigated macOS sandbox restrictions for consistent paths and learned a lot about the same.
20	#16205 ●	Fix Traktor Collection Access in macOS Sandbox	Resolved access to Traktor's `collection.nml` in sandboxed environments via path reconstruction.
21	#16237 ●	Fix Rekordbox USB Access in macOS Sandbox	Resolved multi-USB sandboxing restrictions for Rekordbox databases on macOS by utilizing security-scoped bookmarks.

Apart from GitHub, I have been active on Zulip engaging with the community and helping out other users in my free time and aim to continue to do so.

Project Objectives

Problem Statement: The Need for DJ-Intelligence (DJ-I)

Modern DJ software relies heavily on accurate metadata: BPM timelines, musical keys, and Points of Interest (POIs) such as cue points and beatgrids. However, a fundamental problem persists: **there is no shared source of truth**. Every software (Mixxx, Traktor, Rekordbox, Serato) uses its own proprietary analyzers, resulting in inconsistent interpretations of the same file. This lack of a shared standard is what Evelynne terms **DJ-Intelligence (DJ-I)**.

To build DJ-I, we must first accurately identify the audio we are talking about. At present, the industry-standard link between audio fingerprints (AcoustID) and metadata (MusicBrainz) has a critical shortcoming. **Recording is not equal to Mastering.**

- **Recording**: The abstract idea of a song as captured in a specific session (e.g., "Bohemian Rhapsody").
- **Mastering**: A specific technical output of that recording for a product (e.g., the 1975 original vinyl, the 1991 CD remaster, or a 2011 "24-track" high-res release).

Understanding the AcoustID Ecosystem

To appreciate why CMRT is necessary, one must understand how AcoustID functions today. The process involves three key actors:

1. **Submission**: When a user fingerprints a file (e.g., via Picard or Mixxx), they submit both the audio fingerprint and the local metadata (Title, Artist, Album).
2. **Database Entry**: If a new, unique fingerprint is added to the AcoustID database, volunteers manually link it to a MusicBrainz Recording ID (MBID).
3. **The Historical Mistake**: In the early design of AcoustID, to avoid fragmentation, all fingerprints that resolved to the same "song" were grouped under a single Recording entry. While this simplified metadata tagging, it ignored the technical reality of **mastering variants**.

This project addresses this by introducing the infrastructure for **mastering-aware sub-clustering**, effectively fixing this historical oversight within the Mixxx context and paving the way for broader MetaBrainz adoption.

The consequence: cue points and beatgrids set on a lossless FLAC remaster **do not transfer correctly** to a standard MP3 release of the same song. This project, alongside the synergy project *"Graphical representation of BPM and Musical Keys in the waveform"*, aims to build the foundation for a mastering-aware metadata standard.

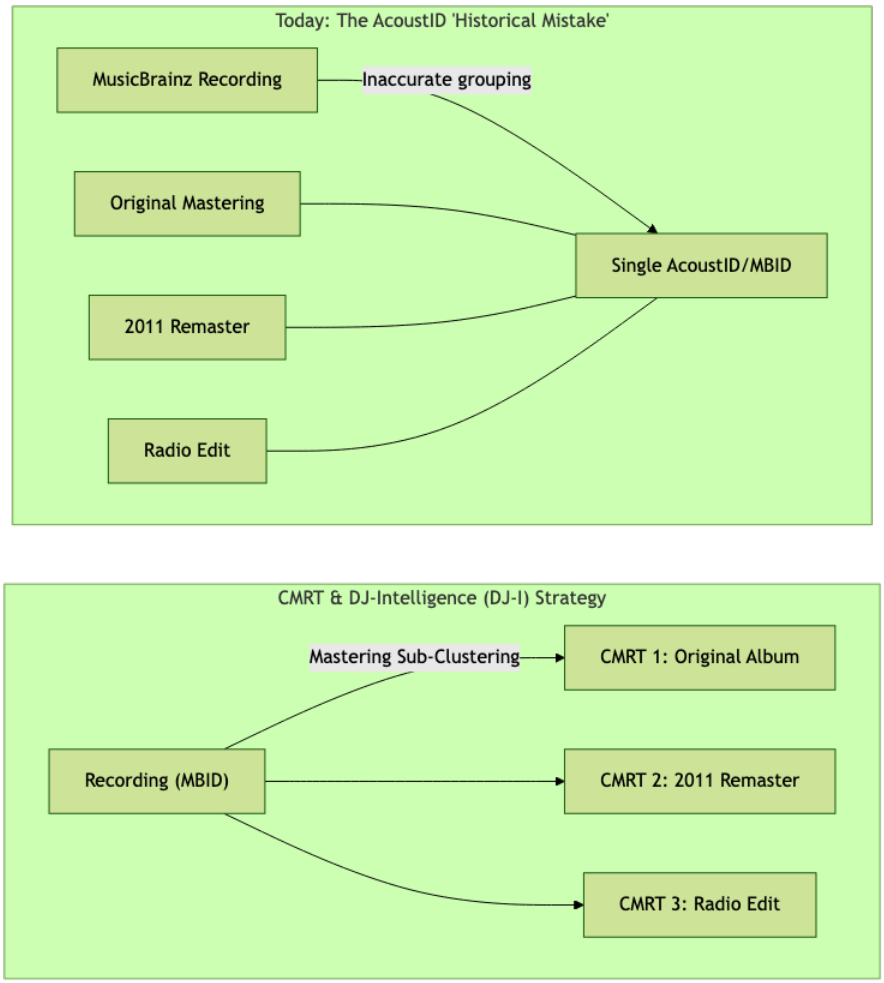

Figure 1: [Top] Today's AcoustID limitation: multiple masterings collapse to a single MBID. [Bottom] CMRT introduces sub-clusters, one per unique mastering.

Mixxx currently uses Chromaprint transiently for lookups. The raw fingerprint data is immediately discarded, leaving no local infrastructure to identify these mastering-specific differences.

Proposed Solution: Introducing CMRT

I propose implementing the **Canonical Master Release Track (CMRT)** system within Mixxx.

Definition of CMRT: A CMRT is a mastering-aware "Source of Truth" for a recording. It identifies one specific, highest-quality version of a track as the **Canonical** reference. Every other version of that same mastering found in a DJ's library is linked to this reference via a calculated timing offset.

By using the **complete Chromaprint fingerprint** instead of just the first 120 seconds, we can distinguish masterings and provide the backbone for DJ-I metadata interchange. This solution consists of four deliverable phases:

1. **Phase 1 - Fingerprint Infrastructure**: A new `AnalyzerChromaprint` class that computes the **full-track** fingerprint during Mixxx's existing analysis pipeline

and persists it via a new `FingerprintDAO` to a new `track_fingerprints` table in MixxxDB.

2. **Phase 2 - Comparison & Local Features**: A `FingerprintMatcher` to calculate sub-millisecond precision timing offsets (identifying identical masterings across different formats). This enables "Find Doubles" and automatic quality-based canonical selection (e.g., automatically promoting a FLAC over an MP3 as the source of truth).

3. **Phase 3 - Network Integration**: Extending `TagFetcher` to resolve fingerprints to AcoustIDs and MBIDs, with user preferences for anonymous sharing. This prepares Mixxx for future CMRT interchange with MetaBrainz.

4. **Phase 4 - Testing, Polishing & Documentation**: Comprehensive unit and integration testing of all new classes using Mixxx's existing GTest/GMock framework, performance benchmarking, developer documentation, and code review feedback incorporation.

This design is validated by Evelynne's research, which has already produced a prototype database of 62,460 analyzed files with 18,254 canonical references and 8,201 timing comparisons- confirming that full-track fingerprinting is fast (>10 fingerprints/second), that offsets are reliably detectable, and that quality-based canonical selection works in practice.

Technical Implementation

Background: How Chromaprint Works

According to the official AcoustID definition, Chromaprint (originally developed by Lukáš Lalinský) works by converting audio into a sequence of numbers that represent its spectral characteristics. It generates a compact fingerprint by:

1. Resampling to **11,025 Hz mono**
2. Computing a Short-time Fourier Transform (frame size 4096, 2/3 overlap, hop size 1365 samples)
3. Extracting **chroma features** (12 pitch classes) for each frame
4. Applying 16 predefined filters on a 16×12 sliding window to generate **32-bit sub-fingerprints**

The overall fingerprint of a track is comprised of multiple 32-bit sub-fingerprints. Each sub-fingerprint encodes the spectral characteristics of a 124ms window(`hop_time = 1365/11025 ≈ 0.1238s`). Mixxx already links against `libchromaprint` and uses it transiently in `ChromaPrinter` for AcoustID lookups-but only the first 120 seconds, and **the raw data is discarded**.

4.1.1 Detecting Silence with Small Hamming Distances

According to the Hamming Distance principle, we can measure the difference between two binary sub-fingerprints by counting the number of bit positions at which the corresponding bits are different.

In the context of Chromaprint, digital silence manifests as sub-fingerprints with **extremely small Hamming distances** between adjacent frames (very few bits changing over time). By calculating the bitwise difference (`XOR + std::popcount`) between successive 32-bit frames, `AnalyzerChromaprint` can identify these low-entropy segments at the beginning and end of a track. This allows us to definitively detect the "silence pre-gap" and "trailing silence." Once this silence is cropped, we can generate a unique hash of the *actual* audio content—providing a second-level, zero-latency lookup for identical masterings that bypasses encoding noise.

New Class: `AnalyzerChromaprint`

Modeled after `AnalyzerSilence` (simple, stateful, chunk-based processing), this class integrates into Mixxx's existing analysis pipeline. It follows the `Analyzer` abstract interface: `initialize()` → `processSamples()` → `storeResults()` → `cleanup()`.

```
class AnalyzerChromaprint : public Analyzer {
  public:
    explicit AnalyzerChromaprint(UserSettingsPointer pConfig,
                                 QSqlDatabase dbConnection);
    ~AnalyzerChromaprint() override;

    bool initialize(const AnalyzerTrack& track,
            mixxx::audio::SampleRate sampleRate,
```

```
            mixxx::audio::ChannelCount channelCount,
            SINT frameLength) override;
    bool processSamples(const CSAMPLE* pIn, SINT count) override;
    void storeResults(TrackPointer pTrack) override;
    void cleanup() override;

  private:
    bool shouldAnalyze(TrackPointer pTrack) const;
    static void convertSamples(const CSAMPLE* pIn, SINT count,
            QVector<int16_t>& output);

    UserSettingsPointer m_pConfig;
    ChromaprintContext* m_pContext;
    mixxx::audio::SampleRate m_sampleRate;
    SINT m_totalFramesProcessed;
    QVector<int16_t> m_conversionBuffer;
};
```

Listing 1: AnalyzerChromaprint header (proposed)

Key design decisions:

- **No plugin indirection**: Unlike `AnalyzerBeats`, which supports multiple detection plugins, there is exactly one fingerprint library (Chromaprint). Direct usage avoids unnecessary abstraction.
- **No 120-second limit**: Unlike `ChromaPrinter` (`kFingerprintDuration = 120`), this processes **all** chunks the pipeline sends, producing a full-track fingerprint.
- **Negligible memory footprint**: Appending to a `std::vector<int32_t>` for the entire duration of a track represents a very small memory footprint. A 5-minute track is ~2,400 frames. At 4 bytes per frame, the total fingerprint requires less than 10KB, easily fitting within L1 cache.
- **`shouldAnalyze()` skip pattern**: Checks the `track_fingerprints` table before re-analyzing, following the same pattern as `AnalyzerSilence` checking for `N60dBSound` cue existence.
- **Sample conversion**: `chromaprint_feed()` expects `int16_t*` but Mixxx's pipeline provides `CSAMPLE*` (float). A reusable conversion buffer is allocated once per track.

4.2.1 Implementation Sketch: Core Methods

The following shows the critical implementation logic- not syntactically complete, but demonstrating the real data flow through the Chromaprint C API and Mixxx's chunk-based pipeline:

```
bool AnalyzerChromaprint::initialize(
        const AnalyzerTrack& track,
        mixxx::audio::SampleRate sampleRate,
        mixxx::audio::ChannelCount channelCount,
        SINT frameLength) {
    if (!shouldAnalyze(track.getTrack())) {
```

```
        return false; // Already fingerprinted
    }
    m_pContext = chromaprint_new(
            CHROMAPRINT_ALGORITHM_DEFAULT);
    // Start the context for FULL track
    // (no kFingerprintDuration cap!)
    chromaprint_start(m_pContext,
            sampleRate, channelCount);
    m_sampleRate = sampleRate;
    m_totalFramesProcessed = 0;
    return true;
}

bool AnalyzerChromaprint::processSamples(
        const CSAMPLE* pIn, SINT count) {
    // Convert float32 -> int16
    // (reuses SampleUtil::convertFloat32ToS16
    //  from chromaprinter.cpp)
    m_conversionBuffer.resize(count);
    SampleUtil::convertFloat32ToS16(
            m_conversionBuffer.data(), pIn, count);
    // Stream into Chromaprint -- accumulates
    // internally, no 120s limit
    if (!chromaprint_feed(m_pContext,
            m_conversionBuffer.data(), count)) {
        qWarning() << "chromaprint_feed failed"
                   << "at frame"
                   << m_totalFramesProcessed;
        return false;
    }
    m_totalFramesProcessed += count;
    return true;
}

void AnalyzerChromaprint::storeResults(
        TrackPointer pTrack) {
    chromaprint_finish(m_pContext);
    // 1. Extract RAW fingerprint (uint32_t[])
    uint32_t* rawFp = nullptr;
    int rawSize = 0;
    chromaprint_get_raw_fingerprint(
            m_pContext, &rawFp, &rawSize);
    // 2. Extract ENCODED fingerprint (base64)
    char* encoded = nullptr;
    int encodedSize = 0;
    chromaprint_encode_fingerprint(
            rawFp, rawSize,
            CHROMAPRINT_ALGORITHM_DEFAULT,
            &encoded, &encodedSize, 1);
    // 3. Store via FingerprintDAO
    FingerprintDAO::FingerprintData data;
```

```
    data.trackId = pTrack->getId();
    data.encodedFingerprint =
            QString::fromUtf8(encoded, encodedSize);
    data.rawFingerprint = QByteArray(
            reinterpret_cast<const char*>(rawFp),
            rawSize * sizeof(uint32_t));
    data.durationSeconds =
            static_cast<double>(rawSize) * kHopTime;
    data.contentHash = computeContentHash(rawFp, rawSize);
    m_pFingerprintDao->saveFingerprint(&data);
    // 4. Memory cleanup (Chromaprint-allocated)
    chromaprint_dealloc(rawFp);
    chromaprint_dealloc(encoded);
}
```

Listing 2: AnalyzerChromaprint core methods (implementation sketch)

Key Design Decisions:

- **Dual extraction**: We call both `chromaprint_get_raw_fingerprint` (for local comparison) *and* `chromaprint_encode_fingerprint` (for AcoustID submission and compact storage). The existing `ChromaPrinter` only extracts encoded- we need both.
- **RAII context management**: `cleanup()` calls `chromaprint_free(m_pContext)` and nulls the pointer. The destructor also guards against leaks if analysis is cancelled by `AnalyzerWithState`.
- **Memory ownership**: Both `rawFp` and `encoded` are Chromaprint-allocated; they **must** be freed with `chromaprint_dealloc`, not `delete` or `free`. This is a critical requirement for cross-platform stability to avoid heap corruption (particularly Windows CRT mismatch crashes where the library and application use different heaps).
- **No separate audio source opening**: Unlike `ChromaPrinter::getFingerprint()` which opens its own `SoundSourceProxy` and reads separately, `AnalyzerChromaprint` receives chunks from the shared pipeline via `processSamples()` resulting in zero I/O duplication.

Registration in the pipeline requires a single addition to `analyzerthread.cpp` (line 117, after `AnalyzerSilence`):

```
m_analyzers.push_back(AnalyzerWithState(
    std::make_unique<AnalyzerSilence>(m_pConfig)));
// NEW: Full-track fingerprinting for CMRT
m_analyzers.push_back(AnalyzerWithState(
    std::make_unique<AnalyzerChromaprint>(
            m_pConfig, dbConnection)));
```

Listing 3: Pipeline registration

Note that `AnalyzerChromaprint` receives a `dbConnection` parameter (unlike `AnalyzerSilence`) because it writes directly to `track_fingerprints` via `FingerprintDAO`. This follows the same pattern as `AnalyzerWaveform` which also takes a `QSqlDatabase` in its constructor.

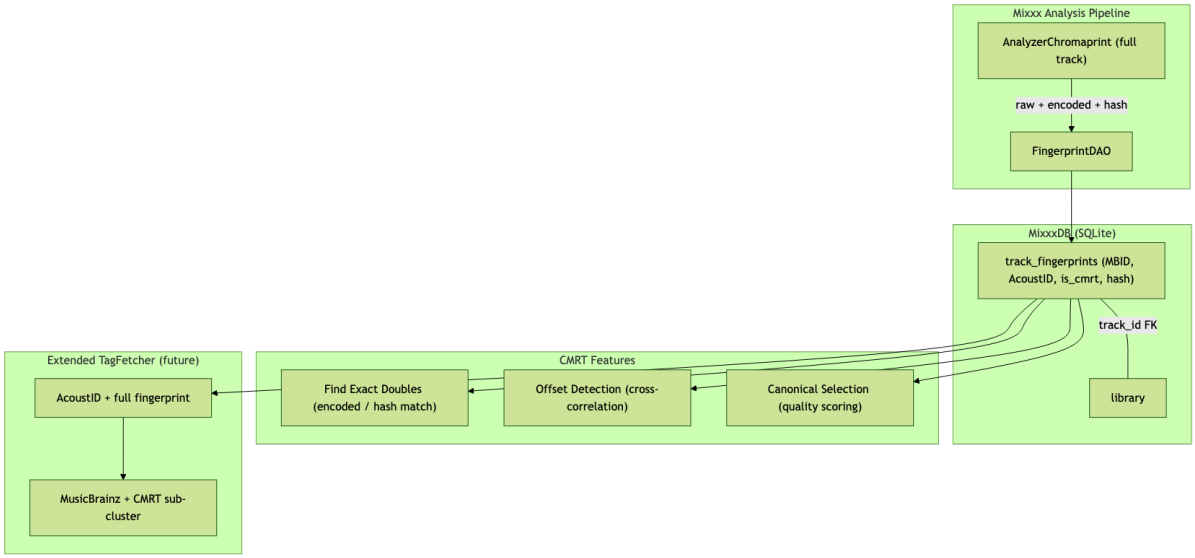

Figure 2: CMRT system architecture: `AnalyzerChromaprint` feeds `FingerprintDAO` which serves local features (duplicate detection, offset calculation, canonical selection) and future network integration with MetaBrainz.

Database Structure

A new table `track_fingerprints` is added to MixxxDB via `SchemaManager` (revision 41 in `res/schema.xml`):

```
CREATE TABLE IF NOT EXISTS track_fingerprints (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    track_id        INTEGER NOT NULL UNIQUE
                        REFERENCES library(id) ON DELETE CASCADE,
    encoded_fp      TEXT NOT NULL,
    raw_fp          BLOB,
    content_hash    TEXT,
    algorithm       INTEGER DEFAULT 1,
    duration_s      REAL,
    fp_date         TEXT,
    acoustid        TEXT,
    mbid            TEXT,
    cmrt_id         TEXT,
    is_cmrt         BOOLEAN DEFAULT 0,
    offset_ms       REAL DEFAULT 0.0,
    offset_conf     REAL DEFAULT 0.0,
    quality_score   REAL DEFAULT 0.0
);
CREATE INDEX idx_fp_acoustid ON track_fingerprints(acoustid);
CREATE INDEX idx_fp_mbid ON track_fingerprints(mbid);
```

Listing 4: Proposed schema (revision 41)

Key Design Decisions:

- `track_id` mirrors Mixxx's existing `track_locations` pattern for perfect data integrity.

- Dual storage: `encoded_fp` (Base64) for interoperability and `raw_fp` (BLOB) for zero-latency local comparisons.
- `offset_ms` and `offset_conf` store the relative time shift and its confidence score for precise mapping.
- `algorithm` and `cmrt_id` ensure future-proofing and distinguish specific masterings.
- Indexes on `acoustid` and `mbid` enable the "Find Doubles" query

New Class: `FingerprintDAO`

Following the `AnalysisDao` pattern, this class provides CRUD operations for the `track_fingerprints` table:

```
class FingerprintDAO : public DAO {
  public:
    struct FingerprintData {
        TrackId trackId;
        QString encodedFingerprint;
        QByteArray rawFingerprint;
        QString contentHash;
        double offsetMs = 0.0;
        double qualityScore = 0.0;
        bool isCmrt = false;
        QString acoustid;
        QString mbid;
    };

    bool saveFingerprint(FingerprintData* pData);
    FingerprintData getFingerprintForTrack(TrackId id) const;
    bool hasFingerprint(TrackId trackId) const;
    QList<FingerprintData> findByAcoustId(
            const QString& acoustid) const;
    FingerprintData findCanonical(
            const QString& acoustid) const;
};
```

Listing 5: FingerprintDAO interface (proposed)

4.4.1 Implementation Sketch: Key SQL Queries

```
bool FingerprintDAO::saveFingerprint(
        FingerprintData* pData) {
    QSqlQuery query(m_database);
    query.prepare(
        "INSERT OR REPLACE INTO track_fingerprints "
        "(track_id, encoded_fp, raw_fp, content_hash,"
        " duration_s, acoustid, mbid, quality_score,"
        " is_cmrt, offset_ms, offset_conf, fp_date) "
        "VALUES (:tid, :enc, :raw, :hash, :dur,"
        " :aid, :mbid, :qs, :cmrt,"
```

```
        " :offset, :conf, datetime('now'))");
    query.bindValue(":hash", pData->contentHash);
    query.bindValue(":cmrt", pData->isCmrt);
    query.bindValue(":tid", pData->trackId.value());
    query.bindValue(":enc",
            pData->encodedFingerprint);
    query.bindValue(":raw",
            pData->rawFingerprint);
    // ... remaining binds ...
    return query.exec();
}
```

Listing 6: FingerprintDAO::saveFingerprint (INSERT OR REPLACE)

```
FingerprintDAO::FingerprintData
FingerprintDAO::findCanonical(
        const QString& acoustid) const {
    QSqlQuery query(m_database);
    query.prepare(
        "SELECT tf.*, tl.filename FROM "
        "track_fingerprints tf "
        "JOIN track_locations tl ON "
        "  tf.track_id = tl.id "
        "WHERE tf.acoustid = :aid "
        "ORDER BY tf.quality_score DESC "
        "LIMIT 1");
    query.bindValue(":aid", acoustid);
    query.exec();
    // ... parse result into FingerprintData ...
}
```

Listing 7: FingerprintDAO::findCanonical (quality-based selection)

4.4.2 Logical Overview: The Voting System

Unlike traditional audio alignment which might look at peaks or loud samples (which are unstable across different encodings), the `FingerprintMatcher` uses a robust **hash-based voting system**:

1. **Robust Feature Selection (Hashing)**: We don't compare the raw audio. We look at the most stable bits of the Chromaprint fingerprint—bits that remain identical even if a track is re-encoded from FLAC to a low-bitrate MP3.

2. **The Shift-and-Vote (Histogram)**: We effectively "slide" one fingerprint over the other and ask: *"If I shift this track by X frames, how many frames match?"*

3. **Finding the Peak**: If hundreds of unique frames all "vote" for the same shift (e.g., +14.2ms), we have found the definitive offset. This method is incredibly robust against noise, silence, and even slight edits because it relies on the "wisdom of the crowd" across thousands of individual frame matches.

4. **Verification (Scoring)**: Once the shift is found, we align the tracks and perform a final check on the Bit Error Rate to ensure the files are indeed the same mastering.

4.4.3 Sub-Millisecond Precision (Daniel's PR #15585)

While Evelynne's original research scripts provided second-level accuracy, this project will adopt the enhanced matcher logic from **Daniel's PR #15585** ([POC] Fingerprint matcher). This implementation achieves sub-1.5ms precision by interpolating across the cross-correlation peaks, ensuring that cues and beatgrids transition seamlessly even between tracks with different codec latencies.

```
struct MatchResult {
    float score;       // 0.0-1.0 similarity
    int offsetFrames;  // Chromaprint frame offset
    double offsetMs;   // Converted to milliseconds
};

// Constants from acoustid_compare.c
constexpr int kMatchBits = 14;
constexpr int kMatchMask = (1 << kMatchBits) - 1;
constexpr int kUniqMask = 1 << (32 - kMatchBits);
// Chromaprint hop time for offset conversion
constexpr double kHopTime = 1365.0 / 11025.0;

inline int matchStrip(int32_t x) {
    return static_cast<uint32_t>(x) >> (32-kMatchBits);
}

MatchResult FingerprintMatcher::compare(
        const int32_t* a, int asize,
        const int32_t* b, int bsize,
        int maxoffset) {
    // Phase 1: Hash-based offset voting
    // Build position index for each
    // top-kMatchBits hash
    std::vector<uint16_t> offsets(
            (kMatchMask + 1) * 2, 0);
    auto* aoff = offsets.data();
    auto* boff = aoff + kMatchMask + 1;
    for (int i = 0; i < asize; i++)
        aoff[matchStrip(a[i])] = i;
    for (int i = 0; i < bsize; i++)
        boff[matchStrip(b[i])] = i;

    // Vote for offsets where hashes match
    int numcounts = asize + bsize + 1;
    std::vector<unsigned short> counts(
            numcounts, 0);
    int topcount = 0, topoffset = 0;
    for (int i = 0; i < kMatchMask; i++) {
        if (aoff[i] && boff[i]) {
            int off = aoff[i] - boff[i];
            if (!maxoffset ||
                    (-maxoffset <= off
                     && off <= maxoffset)) {
                off += bsize;
                counts[off]++;
                if (counts[off] > topcount) {
                    topcount = counts[off];
                    topoffset = off;
                }
```

```
            }
        }
    }
    topoffset -= bsize;

    // Phase 2: Align fingerprints at peak
    const int32_t* pa = a;
    const int32_t* pb = b;
    int as = asize, bs = bsize;
    int minsize = std::min(as, bs) & ~1;
    if (topoffset < 0) {
        pb -= topoffset;
        bs = std::max(0, bs + topoffset);
    } else {
        pa += topoffset;
        as = std::max(0, as - topoffset);
    }
    int size = std::min(as, bs) / 2;
    if (!size || !minsize)
        return {0.0f, topoffset, 0.0};

    // Phase 3: XOR + popcount scoring
    const auto* ad = reinterpret_cast<
            const uint64_t*>(pa);
    const auto* bd = reinterpret_cast<
            const uint64_t*>(pb);
    int biterror = 0;
    for (int i = 0; i < size; i++)
        biterror += std::popcount(ad[i] ^ bd[i]);

    float score = (size * 2.0f / minsize)
        * (1.0f - 2.0f * biterror / (64*size));
    score = std::max(0.0f, score);

    // Diversity penalty (from acoustid_compare.c)
    // Penalizes repetitive/silent fingerprints
    // ... (diversity calculation omitted
    //      for brevity, see full port) ...

    return {score, topoffset,
            topoffset * kHopTime * 1000.0};
}
```

Listing 8: FingerprintMatcher::compare (ported from acoustid_compare.c)

Choosing `maxoffset`: The `maxoffset` parameter is **not** exposed as a user preference,it is a technical boundary, not a setting the user should ever need to adjust. It will be defined as a named constant in `cmrtconversions.h`:

```
// Evelynne's research dataset (8,201 comparisons) showed
// a real-world maximum offset of 44.025s. We use 60s
// (484 frames) as a conservative upper bound with headroom.
// Passing 0 to match_fingerprints2 means "no limit",
```

```
// so we always pass this value explicitly.
constexpr int kDefaultMaxOffsetFrames =
    static_cast<int>(60.0 / kHopTime); // = 484 frames
```

Listing 9: maxoffset constant derived from empirical data

This constant lives alongside the other conversion utilities in `cmrtconversions.h` so it can be adjusted in one place if future data suggests a different bound. Passing it explicitly (rather than 0 = "no limit") keeps comparisons well-scoped and prevents degenerate high-scoring matches on tracks that share identical finales but have wildly different total lengths.

Why this specific algorithm?

- **Linear time**: The hash-voting phase is $O(\texttt{MATCH\_MASK}) = O(16384)$ constant, not a typical bruteforce $O(n \times k)$. For a 5-minute track (∼2,400 sub-fingerprints), this is ∼7× faster than brute-force.
- **Encoding-robust**: `MATCH_STRIP` only uses the top 14 bits of each 32-bit sub-fingerprint. The lower 18 bits are the most affected by lossy encoding artifacts- discarding them is what makes this work across MP3 and FLAC versions of the same mastering.
- **Production-proven**: This exact algorithm has handled **billions** of fingerprint comparisons on AcoustID's PostgreSQL servers since 2011. It is not experimental.
- **Score-compatible**: Because we produce identical scores to the production server, Evelynne's dataset of 8,201 comparisons provides the empirical basis for threshold calibration in Phase 2, no blind guesswork, just analysis of a known distribution.
- **Hardware-accelerated**: `std::popcount` (C++20) compiles to a single `POPCNT` instruction on x86 and `CNT` on ARM, making the scoring phase essentially I/O-bound on cache lines, not compute-bound.

The scoring formula from the AcoustID source is:

$$\texttt{score} = \frac{2 \times \texttt{size}}{\texttt{minSize}} \times \left(1.0 - \frac{2.0 \times \texttt{bitError}}{64 \times \texttt{size}}\right)$$

Strategic Importance: By implementing this specific algorithm in Mixxx, our local offset detection becomes **100% interoperable** with the official AcoustID ecosystem. Most importantly, it produces scoring results that are **directly comparable** to Evelynne's research dataset (8,201 timing comparisons), allowing her existing threshold data to calibrate our software with zero guesswork. We are not implementing an experimental matcher; we are porting the industry-standard production engine to Mixxx.

Quality-Based Canonical Selection

When multiple files share the same AcoustID, the highest-quality version becomes the local "canonical" reference. Following Evelynne's format quality rankings from her research database:

Format	Lossless	Quality Score
FLAC	✓	100
ALAC	✓	100
AIFF	✓	100
WAV	✓	90
M4A	✓	85
AAC		65
MP3		60
OGG		55
WMA		50

Additional modifiers apply: +5 for sample rates above 44,100 Hz, +3 for bit depths above 16-bit. This scoring ensures that lossless "Source of Truth" files are always favored.

4.5.1 Interchange: From Desktop to Portable

Beyond network interchange with ListenBrainz, CMRT is critical for **local DJ metadata interchange**. A DJ preparing a high-res FLAC on their desktop can seamlessly sync the metadata to a portable RPi, tablet, or phone (leveraging Mixxx's new QML-based future) even if the portable device uses lower-bitrate MP3s for storage. The CMRT offset ensures the cues "just work."

4.5.2 Implementation Sketch: `QualityScorer`

The scoring logic is encapsulated in a standalone utility class within a new `mixxx::cmrt` namespace:

```
namespace mixxx::cmrt {

double QualityScorer::baseScoreForFormat(
        const QString& ext) {
    static const QHash<QString, double> kScores = {
        {"flac", 100.0}, {"aiff", 100.0},
        {"aif",  100.0}, {"wav",   90.0},
        {"alac", 100.0}, {"m4a",   85.0},
        {"aac",   65.0}, {"mp3",   60.0},
        {"ogg",   55.0}, {"opus",  55.0},
        {"wma",   50.0},
    };
    return kScores.value(ext.toLower(), 50.0);
}

double QualityScorer::calculateScore(
        TrackPointer pTrack) {
    const QString ext =
        QFileInfo(pTrack->getLocation()).suffix();
    double score = baseScoreForFormat(ext);

    // Bonus for high sample rate
    if (pTrack->getSampleRate() > 44100)
```

```
        score += 5.0;
    // Bonus for larger file (less compression)
    const qint64 fileSize =
        QFileInfo(pTrack->getLocation()).size();
    score += qMin(5.0,
        static_cast<double>(fileSize)
        / (10 * 1024 * 1024));
    return score;
}

} // namespace mixxx::cmrt
```

Listing 10: QualityScorer implementation sketch

4.5.3 Implementation Sketch: `CmrtJsonSerializer`

This class implements the POI interoperability formula concretely, converting CMRT JSON data into Mixxx `CueInfo` objects with the per-track offset applied:

```
QList<mixxx::CueInfo> CmrtJsonSerializer::toCuePoints(
        const CmrtData& cmrtData,
        double trackOffsetMs,
        mixxx::audio::SampleRate sampleRate) {

    QList<mixxx::CueInfo> cues;

    // 1. MainCue = anchor + offset + start_of_music
    double mainCueMs = cmrtData.anchorMs + trackOffsetMs
            + cmrtData.startOfMusicMs;
    cues.append(mixxx::CueInfo(
            mixxx::CueType::MainCue,
            mainCueMs, std::nullopt,
            std::nullopt,
            "CMRT start_of_music",
            std::nullopt,
            mixxx::CueFlag::None));

    // 2. Segment change markers via HotCues
    int hotCueIdx = 0;
    for (const auto& seg : cmrtData.segments) {
        if (seg.type == BpmSegment::Stable)
            continue; // Don't clutter UI
        double localMs = cmrtData.anchorMs + trackOffsetMs
                + seg.startMs;
        QString label = (seg.type
                == BpmSegment::Increase)
            ? QString("BPM %1 -> %2 (accel)")
                .arg(seg.bpm, 0, 'f', 1)
                .arg(seg.endBpm, 0, 'f', 1)
            : QString("BPM %1 -> %2 (decel)")
                .arg(seg.bpm, 0, 'f', 1)
                .arg(seg.endBpm, 0, 'f', 1);
```

```
        cues.append(mixxx::CueInfo(
                mixxx::CueType::HotCue,
                localMs, std::nullopt,
                hotCueIdx++, label,
                std::nullopt,
                mixxx::CueFlag::None));
    }
    return cues;
}
```

Listing 11: CmrtJsonSerializer::toCuePoints: POI formula in action

This is the **core of DJ metadata interchange**: a CMRT JSON retrieved from ListenBrainz is automatically converted into Mixxx cue points, with the per-file offset ensuring correct positioning on the DJ's specific mastering.

4.5.4 The Canonical Replacement Cascade: Illustration

When a user adds a lossless version of a track they already have in MP3, a well-defined cascade executes:

1. **AnalyzerChromaprint** runs on the new FLAC file (on a background thread), producing a full fingerprint
2. **FingerprintDAO** saves the fingerprint; AcoustID lookup yields ``abc123''
3. **findByAcoustId("abc123")** discovers the existing MP3 entry
4. **FingerprintMatcher::compare()** calculates offset = +0.23s, confidence = 0.72
5. **QualityScorer**: FLAC scores 100.0 vs. MP3 at 60.0 ⇒ **FLAC becomes canonical**
6. **Offset recalculation**: The MP3 record gets `offset_ms = −0.23` (relative to new FLAC canonical). The FLAC record gets `offset_ms = 0.0` (it *is* the canonical). Any future files sharing AcoustID `"abc123"` are always compared against the FLAC.
7. **UI Refresh (Thread Safety)**: The DAO emits a Qt signal (e.g., `trackFingerprintsUpdated()`) so the "Find Doubles" library view or the deck waveforms can update seamlessly, ensuring thread-safe synchronization between the background analysis thread and the main GUI thread.

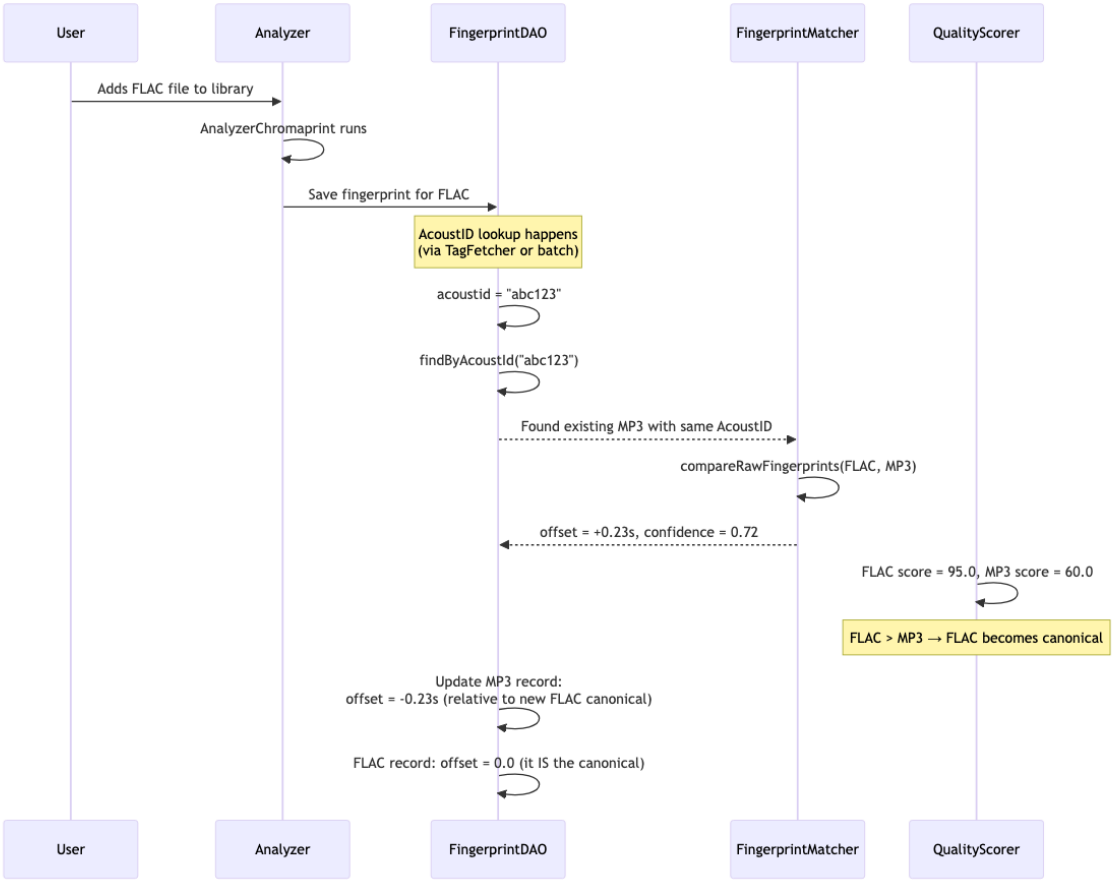

Figure 3: Canonical Replacement Cascade: when a higher-quality file is added, it automatically becomes the canonical reference and all offsets are recalculated.

This cascade is fully automatic and invisible to the user. The practical benefit: when CMRT JSON data later arrives from ListenBrainz, the POI formula uses the canonical FLAC as reference, ensuring the highest-fidelity position mapping for all files.

Coordinate System Conversions

One of the trickier aspects of this project is converting between three different position systems:

- **Chromaprint frame index**: integer, each frame $\approx$ 124ms
- **Milliseconds**: used in CMRT JSON and `CueInfo`
- **Mixxx `FramePos`**: audio frames at the track's native sample rate

The conversion formulas (from Evelynne's Research):

$$\texttt{ms} = \texttt{fp\_frame} \times \frac{1365}{11025} \times 1000 \tag{1}$$

$$\texttt{FramePos} = \frac{\texttt{fp\_frame} \times 1365 \times \texttt{track\_samplerate}}{11025} \tag{2}$$

These are implemented as `constexpr inline` functions in a `cmrtconversions.h` header:

```
namespace mixxx::cmrt {

constexpr int kFpSampleRate = 11025;
constexpr int kFpHopSize = 1365;
constexpr double kHopTime =
    static_cast<double>(kFpHopSize)
    / kFpSampleRate; // = 0.12380952...

inline double fpFrameToMs(int frame) {
    return frame * kHopTime * 1000.0;
}

inline int msToFpFrame(double ms) {
    return static_cast<int>(
        ms / (kHopTime * 1000.0));
}

inline mixxx::audio::FramePos fpFrameToMixxxPos(
        int fpFrame,
        mixxx::audio::SampleRate trackSr) {
    double samplePos =
        static_cast<double>(fpFrame)
         * kFpHopSize * trackSr / kFpSampleRate;
    return mixxx::audio::FramePos(samplePos);
}

/// POI formula implementation:
///   local_ms = (anchor_frame + offset_frames
///               + relative_poi_frame)
///               * hop_time * 1000
inline double applyPOIFormula(
        int anchorFrame, int offsetFrames,
        int relativePOIFrame) {
    return (anchorFrame + offsetFrames
           + relativePOIFrame)
           * kHopTime * 1000.0;
}

} // namespace mixxx::cmrt
```

Listing 12: cmrtconversions.h (utility header)

Integration with Existing Mixxx Systems

Several existing Mixxx features map directly to CMRT concepts:

Feature	Mixxx Implementation	CMRT Mapping
Audio start detection	`AnalyzerSilence` → `N60dBSound` cue	= CMRT `start_of_audio`
BPM detection	`AnalyzerBeats`	Single BPM; CMRT adds segments
Cue system	`CueInfo`, `CueDAO`	POI storage and display framework
Audio resampling	`AudioSourceStereoProxy`	Stereo→mono for fingerprinting
DB migration	`SchemaManager`	Add revision 41 with new table

Notably, `N60dBSound` (CueType 8), already computed by `AnalyzerSilence`, represents the first occurrence of audible sound- this is *exactly* the `start_of_audio` concept. Similarly, `MainCue` maps to `start_of_music`, and `CueInfo` already stores positions in milliseconds, matching the CMRT JSON format with no conversion needed.

Prior Art and Strategic Context

CMRT fills a gap that no existing tool addresses:

Tool	Full FP	Stores FP	Masterings	DJ Meta	Open
AcoustID	120s	No	No	No	Yes
Picard	120s	No	No	No	Yes
beets	120s	Yes	No	No	Yes
Rekordbox	No	N/A	No	Prop.	No
OneLibrary	No	N/A	No	Prop.	No
Mixxx+CMRT	**Full**	**Yes**	**Yes**	**Yes**	**Yes**

CMRT would be the **first open standard** for sharing DJ-specific metadata (BPM segments, key segments, cue points) between DJs with different versions of the same mastering of a recording.

Where CMRT Fits in the MusicBrainz Hierarchy

Understanding the MusicBrainz data model is essential for seeing why CMRT solves a real gap:

- **Release Group** = the abstract "album" (e.g., "Abbey Road by The Beatles")
- **Release** = a specific product (e.g., "Abbey Road, Japanese pressing, 2009 remaster")
- **Track** = position on a specific release's medium
- **Recording** = unique audio content. Different mixes ARE different Recordings, but **different masterings of the same mix share one Recording** (same MBID)

This is the **exact gap CMRT fills**: MusicBrainz considers different masterings of the same mix to be the same Recording. CMRT adds *sub-clustering within a Recording* to distinguish them.

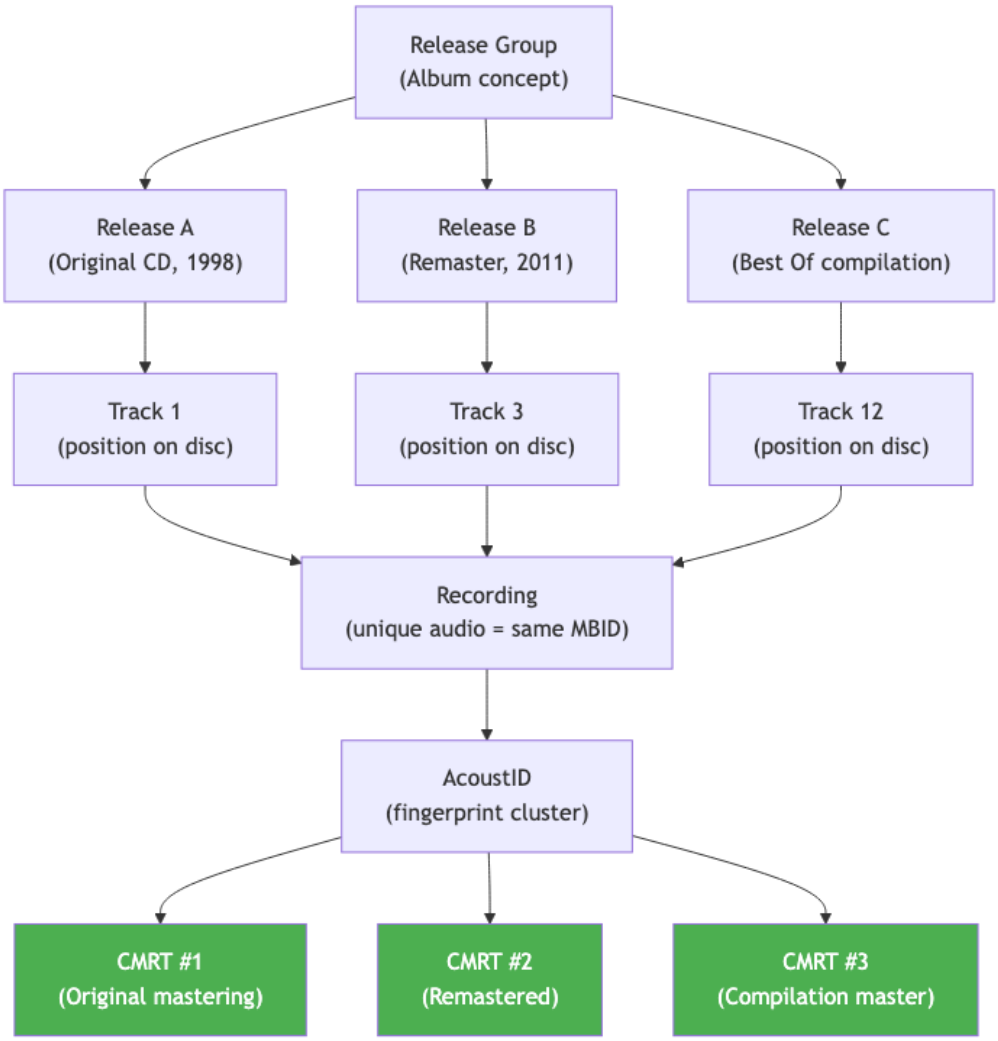

Figure 4: MusicBrainz data hierarchy: Release Group → Release → Track → Recording → AcoustID → CMRT. The green nodes show where CMRT adds mastering sub-clusters.

Strategic Positioning

CMRT extends, not replaces, the existing MetaBrainz ecosystem:

- **Chromaprint** generates fingerprints → CMRT extends this to full-track (not just 120s)
- **AcoustID** clusters fingerprints by recording → CMRT adds mastering sub-clusters within each cluster
- **MusicBrainz** stores recording metadata → CMRT adds structural metadata (BPM segments, POIs)
- **ListenBrainz** tracks listening data → CMRT adds crowd-sourced BPM timelines and shared cue points

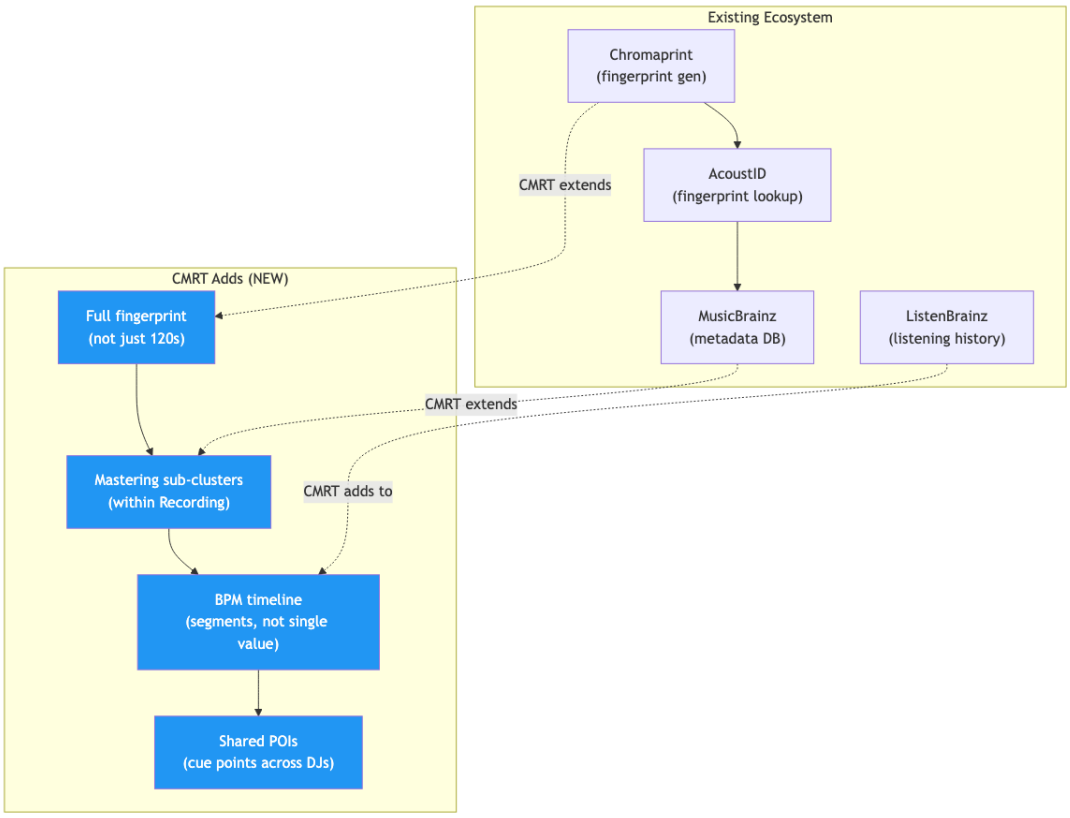

Figure 5: CMRT's strategic positioning: extends the existing MetaBrainz ecosystem (Chromaprint, AcoustID, MusicBrainz, ListenBrainz) with full fingerprinting, mastering sub-clusters, BPM timelines, and shared POIs.

A Concrete Use-Case: "Find Doubles" (Mastering-Aware)

"Find Doubles", detecting and managing duplicate tracks across a DJ's library, is one of the most requested features in DJ software and represents CMRT's most immediate, user-facing value. Notably, **Mixxx currently has zero duplicate detection capability**: no audio fingerprint comparison, no tag-based deduplication, and no way to identify that an MP3 and a FLAC in the library are the same song. A similar request exists from 2017.

4.11.1 Competitive Landscape

Tool	Method	Limitations	Mastering-Aware?
Rekordbox 7.2.8	Audio fingerprint (first 120s only)	Skips tracks <15s or >15min; proprietary	No
Lexicon DJ	Proprietary audio signatures + tag matching	15s–15min; closed-source	No
MediaMonkey	File hash + tag comparison	No audio analysis; misses re-encoded files	No

Tool	Method	Limitations	Mastering-Aware?
Mixxx + CMRT	**Full-track Chromaprint + AcoustID scoring**	**None**	**Yes**

4.11.2 CMRT's Advantage: Mastering-Aware Duplicate Detection

Existing tools answer: "Are these the same song?" CMRT answers a harder question: "Are these the same *recording*, and if so, are they the same *mastering*?" This distinction matters for DJs because:

- Different masterings have **different loudness levels**, affecting gain staging and transitions
- Remasters often have **different intros/outros**, affecting mix points
- Lossy re-encodes accumulate **encoding artifacts** that degrade sound quality
- The DJ may **not know** they have the same track in three different formats across different folders

Because CMRT stores both the AcoustID (which clusters all versions of a recording) and the fingerprint offset/quality score (which distinguishes masterings), the "Find Doubles" UI can present results in a meaningful hierarchy:

1. **Group by AcoustID**: All tracks sharing the same AcoustID are the "same song"
2. **Sub-group by offset**: Within a group, tracks with offset ≈ 0.0 are the same mastering; tracks with significant offsets are different masterings
3. **Rank by quality**: Within each sub-group, the `QualityScorer` highlights the best version
4. **Suggest action**: "Keep FLAC (95 pts), remove MP3 (60 pts)?" or "These are different masterings- keep both?"

4.11.3 Implementation: The Core SQL Query

The "Find Doubles" query is a single SQL statement leveraging the `track_fingerprints` table and its `acoustid` index:

```
SELECT
    tf.acoustid,
    COUNT(*) AS num_versions,
    GROUP_CONCAT(tl.filename, '|') AS files,
    MAX(tf.quality_score) AS best_quality,
    MIN(tf.quality_score) AS worst_quality
FROM track_fingerprints tf
JOIN track_locations tl ON tf.track_id = tl.id
WHERE tf.acoustid IS NOT NULL
    AND tf.acoustid != ''
```

```
GROUP BY tf.acoustid
HAVING COUNT(*) > 1
ORDER BY num_versions DESC;
```

Listing 13: Find Doubles

This query runs in $O(n)$ over the indexed `acoustid` column and returns all groups of duplicate tracks, annotated with their quality spread. For a typical DJ library of 10,000 tracks, this executes in <50ms on SQLite.

4.11.4 Integration with Mixxx Library UI

The "Find Doubles" feature would integrate naturally with Mixxx's existing library sidebar. The implementation follows the pattern of existing library features like `playlistfeature` and `cratefeature`:

- A new sidebar entry "Find Doubles" under the Library section
- Clicking it populates the track table with grouped duplicate results
- Right-click context menu: "Keep This Version", "Remove Duplicate", "Compare Masterings"
- "Compare Masterings" opens a diff-view showing offset, quality scores, and format differences
- Bulk action: "Keep best quality for all groups" with undo support

4.11.5 UI Mockups: Find Doubles Vision

To demonstrate the proposed integration, I have developed a set of UI mockups that show how the CMRT clustering will appear to Mixxx users.

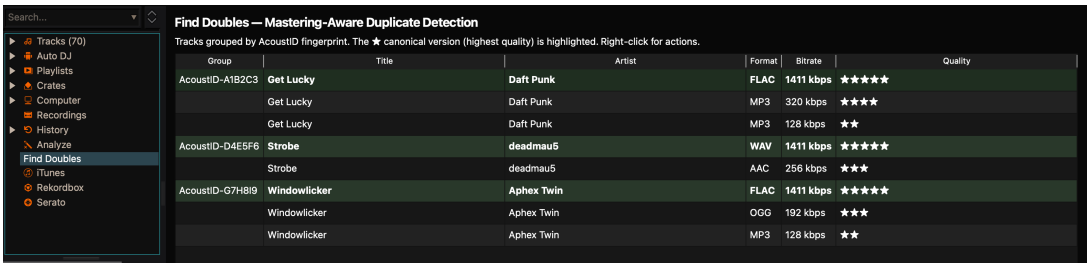

Figure 6: The "Find Doubles" library view showing tracks clustered by AcoustID. Note the green-highlighted rows indicating the automatically selected *canonical* (highest quality) mastering for each group.

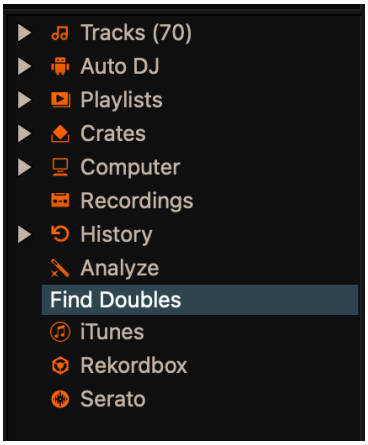

Figure 7: Seamless sidebar integration within the existing library architecture.

Figure 8: Context menu for individual track management within a CMRT cluster.

This is an entirely **offline feature**. It requires no network access. It works from the moment a user's library has been analyzed by `AnalyzerChromaprint` and their tracks have been resolved to AcoustIDs (via the existing `TagFetcher` pathway or a future batch submission feature).

Proof of Concept: Evelynne's Research Database

This project is not speculative. Evelynne has already built a complete research prototype, validating every core assumption with real-world data.

Research Database Summary

Table	Records	Purpose
`analyzed_files`	**62,460**	Every file analyzed, with AcoustID, fingerprint, offset
`canonical_references`	**18,254**	Best-quality file per AcoustID (the "canonical" track)
`timing_comparisons`	**8,201**	Offset calculations between canonical and other files
`analysis_sessions`	**133**	Batch analysis run metadata
`format_quality_rankings`	**9**	Quality scores by audio format

The research scripts include the Queen Mary BPM analyzer (`qmbpm.py`, 823 lines), which performs contiguous BPM timeline analysis using Savitzky-Golay smoothing, produces STABLE/INCREASE/DECREASE segment detection, and represents a track like *Bohemian Rhapsody* as **33 distinct tempo segments** rather than a single BPM value, illustrating what "DJ-Intelligence" (DJ-I) means in practice.

Real-World Offset Statistics (8,201 Comparisons)

All timing comparisons use the **chroma cross-correlation** alignment method, confirming reliable offset detection at scale:

Metric	Value
Average offset	0.076 seconds
Minimum offset	−2.786 seconds
Maximum offset	+44.025 seconds
Average confidence	0.50

The **44-second maximum offset** is significant, it represents versions with extended intros or different edits. Our `FingerprintMatcher` therefore supports configurable maximum offsets.

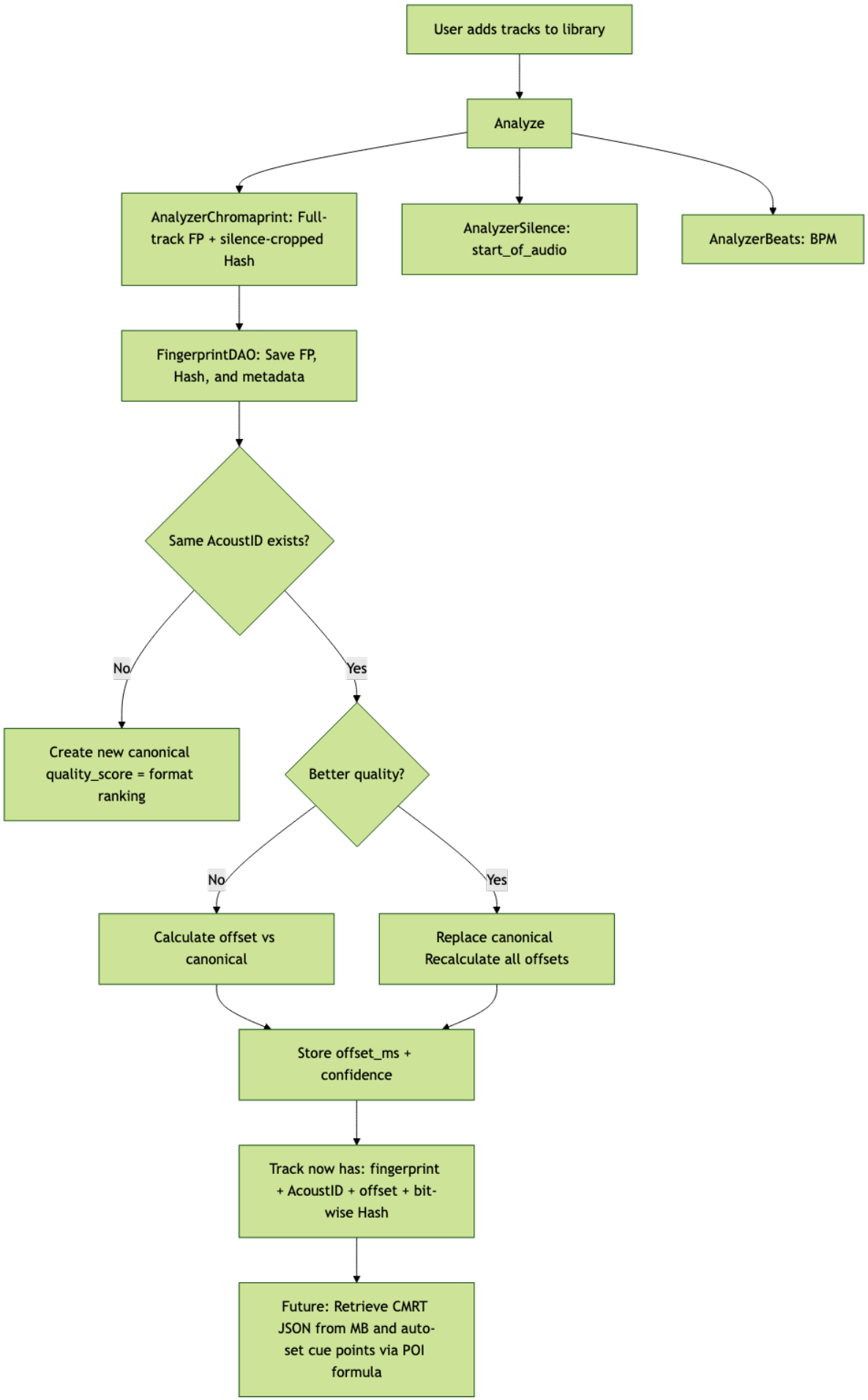

Figure 9: End-to-end CMRT workflow: from track analysis through fingerprint storage,

Alignment Method Comparison (Tested by Evelynne)

Method	Pro	Con
Window matching (XOR)	Direct; uses existing fingerprint data	Brittle for lossy formats
Loudest sample anchor	Fast, simple	Unstable in lossy formats; artifacts shift peak
Chroma cross-correlation (winner)	Most robust for same published versions	Requires harmonic content processing

Mentor-Confirmed Design Decisions

The following were confirmed directly by Evelynne during mentor discussions:

Decision	Eve's Answer	Impact
Should `AnalyzerChromaprint` run by default?	"Yes, enabled by default"	Keep it enabled by default with preference toggle
Is fingerprinting fast enough?	">10 raw fingerprints/sec"	Performance is not a blocker
AcoustID submission rate limit?	3 submits/sec max	Rate-limiter needed for network portion

CMRT Data Format & POI Interoperability

CMRT v1.0 JSON Specification (from Evelynne's Research)

CMRT is defined as a **JSON structure** describing a recording's complete temporal structure. This is the interchange format between Mixxx and the MetaBrainz ecosystem:

```
{
    "recording_mbid": "UUID",
    "start_of_audio": 1880,
    "start_of_music": 3120,
    "segments": [
        { "start": 3120, "bpm": 98.2 },
        { "start": 49180, "end": 56018,
          "bpm": 110.3, "end_bpm": 120.8 }
    ]
}
```

BPM Segment Model (BPMSP)

Each segment encodes a region of the track with potentially changing tempo:

Field	Type	Description
`start`	int (ms)	Start position of this BPM segment
`end`	int (ms)	End position (absent = continues to next segment)
`bpm`	float	BPM value at start
`end_bpm`	float	BPM at end (when present, tempo changes *linearly*; encodes accelerando/ritardando)

POI Interoperability Formula

This is the core formula that makes CMRT data portable across different masterings. Two DJs with different versions of the same mastering of a recording can share cue points using:

$$\texttt{POI\_local} = \texttt{Anchor} + \texttt{offset} + \texttt{relative\_position\_of\_POI}$$

Where **Anchor** is a reference point in the canonical recording, **offset** is the per-file time shift stored in `track_fingerprints.offset_ms`, and **relative_position_of_POI** is the cue position relative to the anchor in the CMRT JSON.

Mapping CMRT to Mixxx's Cue System

Mixxx's existing `CueType` system in `cueinfo.h` maps directly onto CMRT concepts so no new storage primitives are required:

CueType	Value	CMRT Mapping
`HotCue`	1	Maps to CMRT POIs (shared cue points)
`MainCue`	2	Maps to CMRT `start_of_music`
`Beat`	3	Unused in codebase; potential CMRT beat markers
`Loop`	4	Maps to CMRT segment boundaries
`Intro`	6	Maps to CMRT `start_of_music`
`Outro`	7	Maps to end of last CMRT segment
`N60dBSound`	8	**= CMRT `start_of_audio`**

Note that `N60dBSound` (CueType 8) is already computed for every track by `AnalyzerSilence`, and is *exactly* Evelynne's `start_of_audio` concept- requiring zero additional code to reuse. `CueInfo` also stores positions as `std::optional<double>` milliseconds, directly compatible with the CMRT JSON format.

Future Research & Open Questions

While the core implementation is well-defined, the GSoC period will also surface several questions that require empirical investigation:

1. **Graceful AcoustID failure handling**: When a track cannot be resolved to an AcoustID (unreleased tracks, DJ edits, mashups), how should the system behave? We will design a fallback path that still stores the local fingerprint for future pairwise comparison, even without an AcoustID cluster.

2. **Canonical reference UX**: When a higher-quality file automatically replaces the canonical reference, how should the user be informed? We will prototype non-intrusive notification patterns (e.g., library column badge, status bar message) and validate with community feedback on Zulip/Discourse.

3. **Pitch-shifted and tempo-adjusted tracks**: DJs sometimes possess tracks that have been pitch-shifted or tempo-adjusted (e.g., vinyl rips at incorrect speed). The Chromaprint algorithm is partially robust to these transformations, but the offset calculation may produce misleading results. We will characterize the accuracy boundary and document it as a known limitation.

4. **Mastering vs. encoding artifact threshold**: Using Evelynne's dataset of 8,201 timing comparisons, we will empirically determine the score threshold that distinguishes a genuinely different mastering from a mere lossy encoding of the same track. The methodology will involve plotting the score distribution of known same-mastering pairs vs. known different-mastering pairs, and selecting a cutoff that minimizes false positives (unrelated versions treated as identical) while keeping false negatives (missed same-mastering matches) acceptable. Setting the threshold too low risks polluting a user's library with incorrect metadata transfers; setting it too high risks silently missing valid doubles. Finally, I recognize that since the current dataset is primarily focused on 1970s–90s European pop and rock, there is some uncertainty regarding how these thresholds will generalize to other DJ-centric genres like underground electronic music or classical. I intend to discuss with mentors

whether a more diverse supplementary dataset is required and, if so, I am committed to assisting in its preparation (e.g., via targeted sampling of MusicBrainz test recordings) to ensure CMRT robustness across all musical styles.

Testing Strategy & Continuous Integration

All new classes will be tested using Mixxx's existing **GTest/GMock** framework. Tests reside in `src/test/` (using `find_package(GTest)` and `gtest_discover_tests()` in `CMakeLists.txt`) and are executed automatically in GitHub Actions CI on every pull request.

- **Unit tests**: `AnalyzerChromaprintTest` (fingerprint generation from pre-recorded WAV fixtures), `FingerprintDAOTest` (CRUD operations on an in-memory SQLite database), `FingerprintMatcherTest` (known-score pairs verified against AcoustID's production results), `QualityScorerTest` (format → score mapping).
- **Integration tests**: Full pipeline tests (analyze → store → match → canonical selection) using real audio files from the `src/test/id3-test-data/` directory.
- **Performance benchmarks**: Measure fingerprinting throughput (tracks/second) and matcher comparison time across library sizes of 1k, 10k, and 50k tracks to validate the <50ms query target.

Gap Analysis: What Exists vs. What Must Be Built

Requirement	Exists?	Action
Full-track Chromaprint	No (only 120s)	New `AnalyzerChromaprint`
Fingerprint persistence	No (ephemeral)	New DB table + `FingerprintDAO`
MBID per track	Compile-flag only	Store independently in new CMRT table
Offset calculation	No	`FingerprintMatcher` (AcoustID algorithm)
Duplicate detection	No	Compare encoded fingerprints via `findByAcoustId`
Canonical track selection	No	Quality scoring from Evelynne's format rankings
BPM timeline segments	No (single value)	Future: port Queen Mary BPM (`qmbpm.py`) logic to C++
User preference for sharing	No	New preference toggle in `src/preferences/`
MBDB sub-clustering	No (out of scope)	Future MetaBrainz project

Timeline & Milestones

Week	Dates	Phase	Deliverables
1–2	May 25 – June 07	**Phase 1a:** Core Analyzer	• `AnalyzerChromaprint` class (header + implementation) • Sample conversion (`CSAMPLE` → `int16_t`) • Register in `analyzerthread.cpp` • Unit tests: fingerprint generation for test audio files
3–4	June 08 – June 21	**Phase 1b:** Database Layer	• `track_fingerprints` schema (revision 41 in `schema.xml`) • `FingerprintDAO` class with full CRUD • `shouldAnalyze()` skip logic • Integration test: analyze → store → retrieve
5–6	June 22 – July 05	**Phase 2a:** Comparison Engine	• `FingerprintMatcher` with AcoustID's production algorithm • Hash-based histogram offset detection • XOR + popcount scoring • Coordinate system conversions (`cmrtconversions.h`)

Week	Dates	Phase	Deliverables
7	July 06 – July 12	**Phase 2b:** Local Features	• "Find Doubles" query (via `findByAcoustId`) • Quality-based canonical selection • Canonical replacement cascade logic
Mid-term evaluation checkpoint			
8	July 13 – July 19	**Vacation**	Planned break. Will remain available on Zulip for async questions.
9–10	July 20 – Aug 02	**Phase 3:** Network Integration	• Store AcoustID and MBID from `TagFetcher` lookups • User preference: opt-in for anonymous fingerprint sharing • Rate-limited AcoustID submission (3/sec max)
11–12	Aug 03 – Aug 16	**Phase 4:** Testing, Polishing & Docs	• Unit tests for all new classes • Integration test: full pipeline end-to-end • Performance benchmark: full-track fingerprinting time • Code review feedback incorporation • Developer documentation • Buffer for unexpected complexities
13	Aug 17 – Aug 23	**Final Week**	• Final code cleanup and formatting • Submission preparation and final evaluation write-up

Week	Dates	Phase	Deliverables
Final evaluation			

Expected outcomes upon completion:

1. A new `track_fingerprints` table in MixxxDB with complete Chromaprint fingerprints
2. Automatic fingerprinting during track analysis (enabled by default)
3. Local duplicate detection based on complete fingerprints
4. Timing offset calculation between different versions of the same mastering of a recording
5. Quality-based canonical version selection

Future features (beyond GSoC scope, but architecturally enabled):

- Add CMRT sub-clustering to MusicBrainz Database (future MetaBrainz integration)
- Expand MusicBrainz lookup in Mixxx to retrieve and save advanced metadata
- Interchange advanced DJ-metadata through MB/LB (BPM segments, musical key timelines, cue points)

Commitments and Availability

My planned schedule is **2–3 hours on average daily**, totaling approximately **15–18 hours per week**. This pace allows me to comfortably achieve the project's target of **175 hours** across the 12-week coding period while maintaining a high quality of work and ensuring consistent daily communication with my mentors. I have full flexibility to increase this as needed during implementation-heavy phases.

- **Communication**: I will maintain **daily communication** with my mentor Antoinne via the Mixxx Zulip channels. Every morning (IST), I will provide a short update covering the previous day's progress, any blockers encountered, and the planned tasks for the current day.
- **Version control**: All work will be developed in feature branches with incremental, well-documented commits. I will open draft PRs early for continuous review.
- **Time zone**: IST (UTC+5:30), with flexibility to adjust meeting times to accommodate mentor availability.
- **Vacation**: One planned vacation week (week 8) is built into the timeline. I will remain reachable on Zulip for asynchronous questions during this week.

Final Notes

The CMRT project is not just a new feature for Mixxx, it is the **first open standard** for sharing DJ-specific metadata across software, devices, and DJs— "DJ Intelligence" as Evelynne coined it. What excites me most is the depth of validated research behind it. Evelynne's work means we are not speculating about feasibility: **the research is done; the engineering remains.**

Spending weeks studying this project has transformed CMRT from a GSoC proposal into a personal commitment to the future of the DJ community. This work is also a tribute to the vision of **Robert Kaye (Mayhem)**, who recently passed away and was instrumental in giving birth to the CMRT concept within the MetaBrainz Foundation. His combined work with Evelynne and Daniel made possible the very data structures we now seek to extend, and I am deeply honored to help carry this part of his legacy into Mixxx.

The server-side integration, officially adding CMRT as a core entity within MusicBrainz, is not within this proposal's initial scope, but it is not something I intend to leave unfinished. I plan to pursue that work independently throughout the year, ensuring the pipeline is only complete when the data can be shared globally. I am not proposing this as a stretch goal; I am stating it as an intention.

Through my existing contributions to Mixxx, I have tried to demonstrate the curiosity and ownership this project demands. I am eager to be the one who translates that research into working code and to see it through to the end.

I extend my sincere thanks to the Mixxx community for being so encouraging and welcoming throughout this process.